

TUTORIAL PASO A PASO

API Simulada con Node.js y Express

De cero a producción: pruébala en tu computadora y publícala en internet con Render, sin omitir ningún detalle.

UPDS

Ing. Jimmy Requena · Septiembre 2026

Qué vas a lograr en este tutorial



Parte 1 — Local

- Preparar el proyecto e instalar las dependencias con npm
- Entender los 5 endpoints del servidor Express
- Levantar la API en tu computadora
- Probar cada endpoint con curl y verificar las respuestas



Parte 2 — Despliegue

- Preparar el código para producción
- Subir el proyecto a un repositorio de GitHub
- Crear y configurar un Web Service en Render
- Publicar tu API con una URL pública y probarla en vivo

Requisitos previos



Node.js 18 o superior

Incluye npm (se instala junto con Node)



Editor de código

Cualquiera sirve; VS Code es una buena opción



Terminal o consola

CMD, PowerShell, Bash o la terminal integrada del editor



Cuenta de GitHub

Gratuita, se crea en github.com



Cuenta de Render

Gratuita, se crea en render.com



Los archivos del proyecto

server.js, package.json y .gitignore (ya entregados)

PARTE 1

Pruébalo en tu computadora

9 pasos para levantar y probar la API en tu máquina local, de principio a fin.

Estructura del proyecto

```
api-simulada/  
├── server.js  
├── package.json  
├── .gitignore  
└── node_modules/    (se crea luego)
```

¿Qué hace cada archivo?

- `server.js` — todo el código de la API: el servidor Express y los 5 endpoints
- `package.json` — nombre del proyecto, dependencias y el script para arrancarlo
- `.gitignore` — le dice a Git qué carpetas/archivos NO subir (como `node_modules`)
- `node_modules/` — se genera automáticamente al instalar las dependencias; nunca se crea a mano

package.json explicado

```
{
  "name": "api-simulada",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "engines": { "node": ">=18.0.0" },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

Claves importantes

- "scripts.start" → permite ejecutar la app con npm start (Render lo usa automáticamente al desplegar)
- "engines.node" → indica la versión mínima de Node.js que necesita el proyecto
- "dependencies" → librerías que se instalan con npm install; aquí solo Express

Instalar las dependencias

```
cd api-simulada  
npm install
```



Solo necesitas ejecutar esto una vez por proyecto (o cada vez que agregues o cambies una dependencia).

¿Qué hace "npm install"?

- Lee la lista de dependencias en package.json
- Descarga Express (y sus propias dependencias) desde el registro de npm
- Crea la carpeta node_modules/ con todo el código descargado
- Genera o actualiza package-lock.json, que fija las versiones exactas

Los 5 endpoints de un vistazo

Método	Ruta	Qué hace
GET	/api/productos	Lista todos los productos
GET	/api/productos/:id	Obtiene un producto por su id
POST	/api/productos	Crea un producto nuevo
PUT	/api/productos/:id	Actualiza un producto existente
DELETE	/api/productos/:id	Elimina un producto



Todos los datos se guardan en un arreglo en memoria: es perfecto para practicar, pero se reinicia cada vez que apagas el servidor.

Código completo — server.js (1/2)

```
const express = require('express');
const app = express();
const PUERTO = process.env.PORT || 3000;

app.use(express.json());

let productos = [
  { id: 1, nombre: 'Teclado mecánico', precio: 120 },
  { id: 2, nombre: 'Mouse inalámbrico', precio: 45 },
  { id: 3, nombre: 'Monitor 24"', precio: 380 },
];
let siguienteId = 4;

// 1) Listar todos
app.get('/api/productos', (req, res) => {
  res.json(productos);
});

// 2) Obtener uno por id
app.get('/api/productos/:id', (req, res) => {
  const producto = productos.find(p => p.id === Number(req.params.id));
  if (!producto) return res.status(404).json({ error: 'No encontrado' });
  res.json(producto);
});
```

Código completo — server.js (2/2)

```
// 3) Crear uno nuevo
app.post('/api/productos', (req, res) => {
  const { nombre, precio } = req.body;
  if (!nombre || precio === undefined)
    return res.status(400).json({ error: 'Faltan datos' });
  const nuevo = { id: siguienteId++, nombre, precio };
  productos.push(nuevo);
  res.status(201).json(nuevo);
});

// 4) Actualizar un producto
app.put('/api/productos/:id', (req, res) => {
  const producto = productos.find(p => p.id === Number(req.params.id));
  if (!producto) return res.status(404).json({ error: 'No encontrado' });
  const { nombre, precio } = req.body;
  if (nombre !== undefined) producto.nombre = nombre;
  if (precio !== undefined) producto.precio = precio;
  res.json(producto);
});

// 5) Eliminar + arrancar servidor
app.delete('/api/productos/:id', (req, res) => { /* ... */ });
app.listen(PUERTO, () => console.log(`API en :${PUERTO}`));
```

Levantar el servidor

```
npm start
```

Salida esperada en la consola:

```
API simulada corriendo en http://localhost:3000
```

- Ejecuta el script "start" de package.json (node server.js)



Deja esta terminal abierta mientras pruebas la API: si la cierras o presionas Ctrl+C, el servidor se detiene.

Probar los endpoints GET

Abre una segunda terminal (deja el servidor corriendo en la primera) y ejecuta:

```
curl http://localhost:3000/api/productos
```

Respuesta:

```
[ { "id": 1, "nombre": "Teclado mecánico", "precio": 120 },  
  { "id": 2, ... }, { "id": 3, ... } ]
```

```
curl http://localhost:3000/api/productos/2
```

Respuesta:

```
{ "id": 2, "nombre": "Mouse inalámbrico", "precio":  
45 }
```

Probar POST (crear) y PUT (actualizar)

```
curl -X POST http://localhost:3000/api/productos \
-H "Content-Type: application/json" \
-d '{"nombre": "Webcam HD", "precio": 90}'
```

```
{ "id": 4, "nombre": "Webcam HD", "precio": 90 }
```

```
curl -X PUT http://localhost:3000/api/productos/1 \
-H "Content-Type: application/json" \
-d '{"precio": 99}'
```

```
{ "id": 1, "nombre": "Teclado mecánico", "precio":  
99 }
```



POST devuelve el código 201 (Created) y el nuevo objeto con su id asignado. PUT solo cambia los campos que envías en el body.

Probar DELETE y validar los errores

```
curl -X DELETE  
http://localhost:3000/api/productos/3
```

```
curl -i http://localhost:3000/api/productos/999
```

```
{ "mensaje": "Producto eliminado",  
  "producto": { "id": 3, ... } }
```

```
HTTP/1.1 404 Not Found  
{ "error": "Producto no encontrado" }
```



Si llegaste hasta aquí y las 9 pruebas respondieron como se muestra: ¡tu API simulada funciona por completo en local! Ya puedes pasar a la Parte 2 para publicarla en internet.

Problemas comunes al probar en local

Problema	Causa probable	Solución
Error: listen EADDRINUSE :::3000	El puerto 3000 ya está en uso por otro proceso	Cierra el proceso anterior o usa otro puerto: PORT=3001 node server.js
Cannot find module 'express'	No se ejecutó "npm install" en la carpeta del proyecto	Corre npm install dentro de api-simulada/ antes de iniciar el servidor
"curl" no se reconoce (Windows)	CMD antiguo sin curl disponible	Usa PowerShell, Git Bash, o una app como Postman / Thunder Client
Todas las rutas devuelven 404	La URL no incluye "/api" o tiene un error de escritura	Verifica que uses exactamente /api/productos

PARTE 2

Publícalo en internet con Render

8 pasos para llevar tu API de tu computadora a una URL pública y accesible desde cualquier lugar.

Preparar el código para producción

Antes

```
const PUERTO = 3000;
```

Después (ya aplicado en tu server.js)

```
const PUERTO = process.env.PORT || 3000;
```

¿Por qué es imprescindible este cambio?

- Render (y la mayoría de plataformas de despliegue) asignan el puerto de forma dinámica a través de la variable de entorno PORT
- Si tu código ignora esa variable y usa siempre 3000, Render no podrá conectarse a tu app y el despliegue falla con "Application failed to respond"
- También confirma que package.json tenga "scripts": { "start": "node server.js" } — Render ejecuta npm start por defecto

Subir el proyecto a GitHub

a) Crea el repositorio en GitHub

- Entra a github.com → botón "New" (o el ícono +) → nombra el repo, por ejemplo api-simulada → Create repository (sin agregar README ni .gitignore, ya los tienes)

b) Sube tu proyecto desde la terminal

```
cd api-simulada
git init
git add .
git commit -m "API simulada lista para desplegar"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/api-simulada.git
git push -u origin main
```

Crear tu cuenta en Render y conectar GitHub

1

Regístrate gratis

Entra a dashboard.render.com/register. Puedes crear la cuenta directamente con tu usuario de GitHub (más rápido) o con correo electrónico.

2

Agrega tus credenciales de Git

Si te registraste con correo: ve a Account Settings → Git Deployment Credentials → Add credential.

3

Autoriza el acceso

Acepta la autorización de Render hacia GitHub. Puedes dar acceso a todos tus repos o solo a api-simulada.

Crear el Web Service



En detalle

- En el Dashboard de Render, haz clic en el botón "+ New" (esquina superior derecha)
- Selecciona "Web Service" en el menú desplegable
- Verás la lista de repositorios a los que le diste acceso: elige api-simulada
- Haz clic en "Connect" junto al repositorio para continuar a la configuración

Configurar el servicio

Campo	Valor recomendado
Name	api-simulada (o el nombre que prefieras)
Region	La más cercana a tus usuarios, ej. Oregon (US West)
Branch	main
Root Directory	Vacío, si el proyecto está en la raíz del repositorio
Language / Runtime	Node
Build Command	npm install
Start Command	npm start
Instance Type	Free



Al terminar de revisar estos campos, baja hasta el final del formulario y haz clic en "Deploy Web Service".

Monitorear el despliegue y obtener tu URL

1

Se inicia el primer build

Render crea el servicio y arranca automáticamente el primer despliegue.

2

Revisa el registro en vivo

En la pestaña "Deploys" puedes ver el log en tiempo real: instalación de dependencias y arranque del servidor.

3

Espera el estado "Live"

Cuando el build termina sin errores, el estado del servicio cambia a "Live": tu API ya está publicada.

4

Copia tu URL pública

Aparece en la parte superior del panel, con el formato `https://api-simulada.onrender.com`

Probar la API ya desplegada

```
curl https://api-simulada.onrender.com/api/productos
```

```
curl -X POST https://api-simulada.onrender.com/api/productos \  
-H "Content-Type: application/json" \  
-d '{"nombre": "Webcam HD", "precio": 90}'
```



Plan gratuito de Render: si nadie usa el servicio durante unos 15 minutos, se "duerme" para ahorrar recursos. La primera petición después de estar inactivo puede tardar hasta ~50 segundos en responder mientras se reactiva — es el comportamiento normal del plan Free, no un error.

Despliegue continuo (CI/CD automático)



Lo que significa para ti

- No necesitas repetir la configuración del servicio en Render
- Cada vez que hagas git push a la rama conectada (main), Render detecta el cambio y dispara un nuevo build y despliegue automáticos
- Puedes seguir el progreso de cada nuevo despliegue en la misma pestaña "Deploys"
- Si un build falla, Render mantiene funcionando la última versión válida hasta que corrijas el error

Problemas comunes al desplegar en Render

Problema	Causa probable	Solución
"Application failed to respond"	La app no usa la variable de entorno PORT	Confirma que server.js tenga <code>const PUERTO = process.env.PORT 3000</code> (ya incluido)
Build failed / "Cannot find module"	Falta una dependencia en package.json, o el push no incluyó todos los archivos	Revisa "dependencies" y que .gitignore excluya solo <code>node_modules/</code> , no el código
La primera petición tarda mucho	El plan gratuito "duerme" el servicio tras inactividad	Espera ~50 segundos en la primera petición, o usa un plan pago para evitarlo
Los cambios no se reflejan	No se hizo push a la rama conectada (main)	<code>git push origin main</code> y revisa que aparezca un nuevo build en "Deploys"

Resumen y próximos pasos

Lo que lograste hoy

- ✓ API funcionando en tu computadora, con 5 endpoints
- ✓ Cada endpoint probado con curl (incluyendo errores 404 y 400)
- ✓ Código subido a un repositorio de GitHub
- ✓ API publicada en internet con una URL propia en Render
- ✓ Despliegue automático configurado con cada git push

Próximos pasos sugeridos

- Reemplazar el arreglo en memoria por una base de datos real (ej. Supabase / PostgreSQL)
- Agregar autenticación a los endpoints que lo requieran
- Mover datos sensibles a variables de entorno en Render
- Sumar pruebas automatizadas (tests) antes de cada despliegue